



មហាវិទ្យាល័យវិស្វកម្ម
FACULTY OF ENGINEERING

Computer Programming

Lecture 4 Values and Variables - 2

Chhoeum Vantha, Ph.D.
Telecom & Electronic Engineering

Contents

- Comments
- Basic Data Types
- and Operations
- Evaluating Expressions and Operator Precedence
- Augmented Assignment Operators
- Increment and Decrement Operators
- Numeric Type Conversions

Comments

- Comments are used in C++ to make the code more readable and to explain what the code is doing. The compiler ignores all comments when the program is compiled.

Comments

Types of Comments:

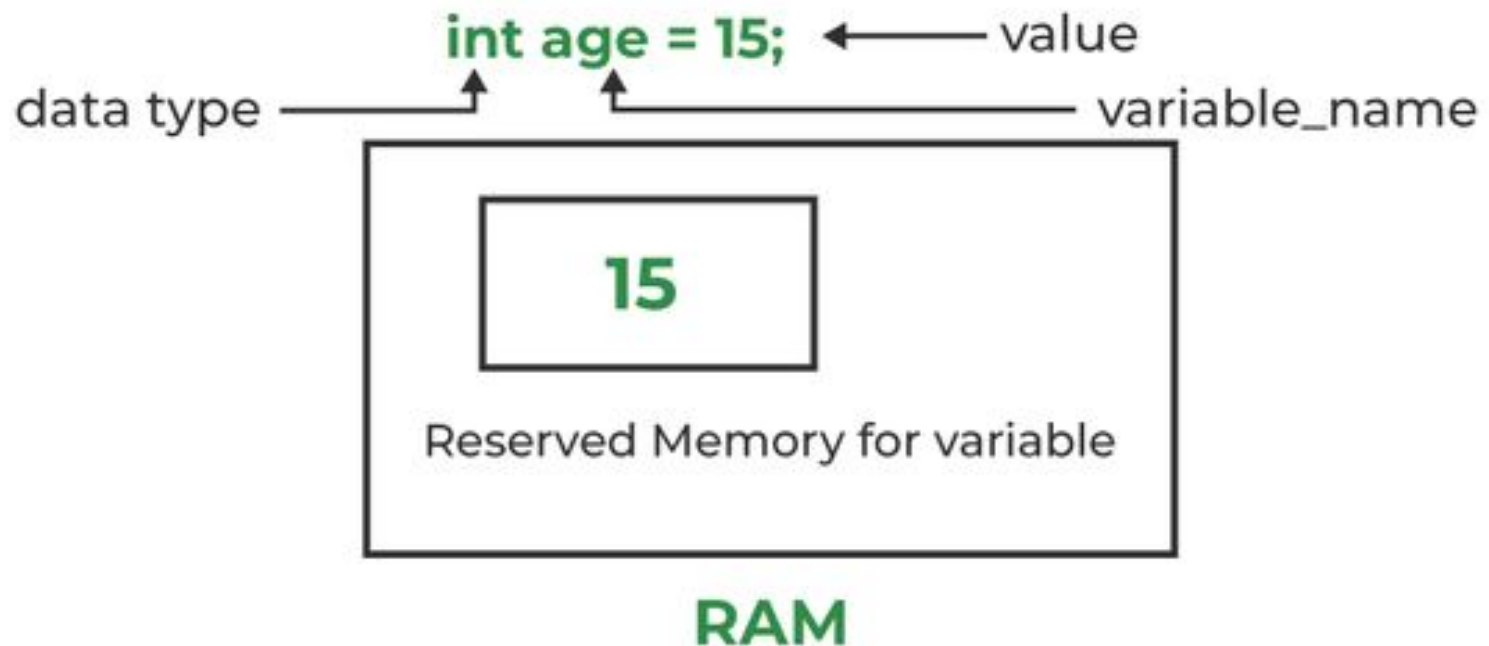
1. Single-line Comment
 - Starts with //
 - Used for short explanations or notes
2. Multi-line Comment
 - Starts with /* and ends with */
 - Used for longer explanations or to temporarily disable code

Comments

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // Declare variables
6      int a = 10;
7      int b = 20;
8
9      /* Calculate the sum of a and b
10     |   and print the result */
11     int sum = a + b;
12     cout << "Sum = " << sum << endl;
13
14     return 0; // Exit the program
15 }
```

Variable

- A variable is a name given to a memory location. It is used to store data that can be changed during program execution.



Basic Data Types

Data Type	Description	Example Values
int	Integer values	-5, 0, 100
float	Floating-point (decimal) numbers	3.14, -0.1
double	Double-precision float	3.14159, 2.71828
char	Single character	'A', 'z', '3'
bool	Boolean (true/false)	true, false
void	No value (used for functions)	-

Basic Data Types

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      // Integer type
6      int age = 20;
7
8      // Floating point type
9      float height = 170.5f;
10
11     // Double precision floating point type
12     double weight = 65.75;
13
14     // Character type
15     char grade = 'A';
16
17     // Boolean type
18     bool isPassed = true;
19
20     // Displaying the values
21     cout << "Age: " << age << endl;
22     cout << "Height: " << height << " cm" << endl;
23     cout << "Weight: " << weight << " kg" << endl;
24     cout << "Grade: " << grade << endl;
25     cout << "Passed Exam: " << isPassed << endl;
26
27     return 0;
28 }
```


Common Type Modifiers

- Signed, Unsigned, Short, long

Modifier	Description	Example Usage
signed	Default for integer types; can store both positive and negative numbers.	signed int a = -10;
unsigned	Only stores non-negative numbers (0 and positive).	unsigned int b = 20;
short	Reduces the size of integer to at least 16 bits.	short int c = 1000;
long	Increases the size of integer to at least 32 bits.	long int d = 100000L;
long long	Further increases size, usually at least 64 bits.	long long int e = 100000000000LL;
const	Declares a constant value that cannot be changed.	const int f = 50;

check the size of basic data types

- *sizeof* operator use to check the size of basic data types in C++

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      cout << "Size of char: " << sizeof(char) << " byte" << endl;
5      cout << "Size of int: " << sizeof(int) << " bytes" << endl;
6      cout << "Size of short int: " << sizeof(short int) << " bytes" << endl;
7      cout << "Size of long int: " << sizeof(long int) << " bytes" << endl;
8      cout << "Size of long long int: " << sizeof(long long int) << " bytes" << endl;
9      cout << "Size of float: " << sizeof(float) << " bytes" << endl;
10     cout << "Size of double: " << sizeof(double) << " bytes" << endl;
11     cout << "Size of long double: " << sizeof(long double) << " bytes" << endl;
12     cout << "Size of bool: " << sizeof(bool) << " byte" << endl;
13
14     return 0;
15 }
```

Operations

- Numeric Operators

<i>Operator</i>	<i>Name</i>	<i>Example</i>	<i>Result</i>
+	Addition	$34 + 1$	35
-	Subtraction	$34.0 - 0.1$	33.9
*	Multiplication	$300 * 30$	9000
/	Division	$1.0 / 2.0$	0.5
%	Modulus	$20 \% 3$	2

Operations

- Numeric Operators

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int seconds;
6      cout << "Enter an integer for seconds: ";
7      cin >> seconds;
8      int minutes = seconds / 60;
9      int remainingSeconds = seconds % 60;
10     cout << seconds << " seconds is " << minutes <<
11     " minutes and " << remainingSeconds << " seconds " << endl;
12     return 0;
13 }
```

Enter an integer for seconds: 500
500 seconds is 8 minutes and 20 seconds

Operations

- Exponent Operations
 - The `pow(a, b)` function can be used to compute a^b .
pow is a function defined in the *cmath* library.

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  int main()
5  {
6      cout << pow(2.0, 3) << endl;
7      cout << pow(4.0, 0.5) << endl;
8      cout << pow(2.5, 2) << endl;
9      cout << pow(2.5, -2) << endl;
10     return 0;
11 }
```

```
8
2
6.25
0.16
```

Evaluating Expressions and Operator Precedence

- Evaluating in the same way as arithmetic expressions.
- Multiplication, division, and remainder operators are applied next.
- If an expression contains several multiplication, division, and remainder operators, they are applied from left to right.
- Addition and subtraction operators are applied last.
- If an expression contains several addition and subtraction operators, they are applied from left to right.

Evaluating Expressions and Operator Precedence

$$\frac{3 + 4x}{5} - \frac{10(y - 5)(a + b + c)}{x} + 9\left(\frac{4}{x} + \frac{9 + x}{y}\right)$$

$$(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x + 9 * (4 / x + (9 + x) / y)$$

$$3 + 4 * 4 + 5 * (4 + 3) - 1$$

(1) inside parentheses first

$$3 + 4 * 4 + 5 * 7 - 1$$

(2) multiplication

$$3 + 16 + 5 * 7 - 1$$

(3) multiplication

$$3 + 16 + 35 - 1$$

(4) addition

$$19 + 35 - 1$$

(5) addition

$$54 - 1$$

(6) subtraction

$$53$$

Evaluating Expressions and Operator Precedence

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int fahrenheit;
6      cout << "Enter a degree in Fahrenheit: ";
7      cin >> fahrenheit;
8      double celsius = (5.0 / 9) * (fahrenheit - 32);
9      cout << "Fahrenheit " << fahrenheit << " is " <<
10     celsius << " in Celsius" << endl;
11     return 0;
12 }
```

Enter a degree in Fahrenheit: 100 
Fahrenheit 100 is 37.7778 in Celsius

Augmented Assignment Operators

- The operators $+$, $-$, $*$, $/$, and $\%$ can be combined with the assignment operator to form augmented operators.

<i>Operator</i>	<i>Name</i>	<i>Example</i>	<i>Equivalent</i>
$+=$	Addition assignment	$i += 8$	$i = i + 8$
$-=$	Subtraction assignment	$i -= 8$	$i = i - 8$
$*=$	Multiplication assignment	$i *= 8$	$i = i * 8$
$/=$	Division assignment	$i /= 8$	$i = i / 8$
$\% =$	Modulus assignment	$i \% = 8$	$i = i \% 8$

Increment and Decrement Operators

- The increment (++) and decrement (--) operators are for incrementing and decrementing a variable by 1.
 - increment operator (++)
 - decrement operator (--)

```
int i = 3, j = 3;  
i++; // i becomes 4  
j--; // j becomes 2
```

```
int i = 3, j = 3;  
++i; // i becomes 4  
--j; // j becomes 2
```

- Both the increment and decrement operators can either precede (prefix) or follow (postfix) the operand.

Increment and Decrement Operators

- differences between the prefix form of ++ (or --) and the postfix form of ++ (or --).

Operator	Name	Description	Example (assume $i = 1$)
<code>++var</code>	preincrement	Increment <code>var</code> by <code>1</code> and use the new <code>var</code> value in the statement	<pre>int j = ++i; // j is 2, i is 2</pre>
<code>var++</code>	postincrement	Increment <code>var</code> by <code>1</code> , but use the original <code>var</code> value in the statement	<pre>int j = i++; // j is 1, i is 2</pre>
<code>--var</code>	predecrement	Decrement <code>var</code> by <code>1</code> and use the new <code>var</code> value in the statement	<pre>int j = --i; // j is 0, i is 0</pre>
<code>var--</code>	postdecrement	Decrement <code>var</code> by <code>1</code> and use the original <code>var</code> value in the statement	<pre>int j = i--; // j is 1, i is 0</pre>

Increment and Decrement Operators

- differences between the prefix form of ++ (or --) and the postfix form of ++ (or --).

```
int i = 10;  
int newNum = 10 * i++;  
cout << "i is " << i  
      << ", newNum is " << newNum;
```

Same effect as

```
int newNum = 10 * i;  
i = i + 1;
```

```
i is 11, newNum is 100
```

```
int i = 10;  
int newNum = 10 * (++i);  
cout << "i is " << i  
      << ", newNum is " << newNum;
```

Same effect as

```
i = i + 1;  
int newNum = 10 * i;
```

```
i is 11, newNum is 110
```

Numeric Type Conversions

- **Floating-point** numbers can be converted into integers using explicit casting.

```
int i = 34.7;      // i becomes 34
double f = i;      // f is now 34
double g = 34.3;   // g becomes 34.3
int j = g;         // j is now 34
```

- Static casting can also be done using the (type) syntax—that is, giving the target type in parentheses, followed by a variable, a literal, or an expression.
- `static_cast<type>(value)`

W4 - Practice

Exercise 1.

Write a program that prompts the user to enter two points (x1, y1) and (x2, y2) and displays their distance between them. The formula for computing the distance is

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Note that you can use `pow(a, 0.5)` to compute

$$\sqrt{a}$$

Here is a sample run:

```
Enter x1 and y1: 1.5 -3.4 ↵ Enter
```

```
Enter x2 and y2: 4 5 ↵ Enter
```

```
The distance between the two points is 8.76413
```

Exercise 2.

Write a program that prompts the user to enter the side of a hexagon and displays its area. The formula for computing the area of a hexagon is

$$Area = \frac{3\sqrt{3}}{2}s^2,$$

- Here is a sample run:

```
Enter the side: 5.5   
The area of the hexagon is 78.5895
```


Exercise 3.

Write a program that displays the following table:

x	y	pow(x, y)
2.5	1.2	3.00281
5.0	2.4	47.5913
1.2	3.6	1.92776
2.4	5.0	79.6262
3.6	2.5	24.5899

Thanks!